注：手写模数

快速幂

```
i64 qpow (i64 a, i64 b)
{
    i64 res = 1;
    while (b) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}
```

组合数

```
const int N = 1e6;

i64 f[N], g[N];

i64 qpow (i64 a, i64 b)
{
    i64 res = 1;
    while (b) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

i64 C (i64 n, i64 m)
{
    return f[n] * g[m] % mod * g[n - m] % mod;
}

void init ()
{
    f[0] = 1, g[0] = 1;
    for (int i = 1; i < N; i++) {
        f[i] = f[i - 1] * i % mod;
        g[i] = qpow(f[i], mod - 2) % mod;
    }
}
```

```
i64 C[35][35];

void init ()
{
    for (int i = 0; i <= 34; i++) {
        C[i][0] = 1, C[i][i] = 1;
        for (int j = 1; j < i; j++) {
            C[i][j] = C[i - 1][j] + C[i - 1][j - 1];
        }
    }
}
```

质数筛

```
const int N = 1e5;

bool vis[N];
int primes[N];//1-based
int cnt = 0;

void get_primes (int n)
{
    for (int i = 2; i <= n; i++) {
        if (!vis[i]) primes[++cnt] = i;
        for (int j = 1; j <= cnt && i * primes[j] <= n; j++) {
            vis[i * primes[j]] = 1;
            if (i % primes[j] == 0) break;
        }
    }
}
```

质因数分解

```
const int N = 1e6 + 1;

bool vis[N];
int primes[N], mnp[N];
int cnt = 0;
vector <vector<pair<int, int> > > v(N);

void get_factors ()
{
    mnp[1] = 1;
    for (int i = 2; i < N; i++) {
        if (!vis[i]) {
            primes[++cnt] = i;//1-based
            mnp[i] = i;
        }
        for (int j = 1; j <= cnt && i * primes[j] < N; j++) {
            vis[i * primes[j]] = 1;
            mnp[i * primes[j]] = primes[j];
            if (i % primes[j] == 0) break;
        }
```

```
    }

    for (int i = 1; i < N; i++) {
        int t = i;
        while (t > 1) {
            int p = mnp[t];
            int count = 0;
            while (t % p == 0) {
                t /= p;
                count++;
            }
            v[i].push_back({p, count});
        }
    }
}
```

并查集

```
class DSU {
    public:
        vector <int> fa, rk, sz;
        int comps; //联通块数
        DSU (int n = 0) {
            init(n);
        }

        void init (int n) {
            fa.resize(n + 1);
            rk.assign(n + 1, 1);
            sz.assign(n + 1, 1);
            iota(fa.begin(), fa.end(), 0LL);
            comps = n;
        }

        int find (int x) {
            return x == fa[x] ? x : (fa[x] = find(fa[x]));
        }

        bool merge (int i, int j) {
            int x = find(i), y = find(j);
            if (x == y) return false;
            if (rk[x] < rk[y]) swap(x, y);
            fa[y] = x;
            sz[x] += sz[y];
            if (rk[x] == rk[y]) rk[x]++;
            comps--;
            return true;
        }

        int size (int x) {
            return sz[find(x)];
        }
};
```

树状数组

```
class BIT {
    public:
        int n;
        vector <i64> bit;

        BIT (int n = 0) {init(n);}

        void init (int m) {
            n = m;
            bit.assign(n + 1, 0);
        }

        static i64 lowbit (i64 x) { return x & -x; }

        i64 sum (int idx) const {
            i64 res = 0;
            for (; idx > 0; idx -= lowbit(idx)) res += bit[idx];
            return res;
        }

        void point_add (int idx, i64 diff) {
            for (; idx <= n; idx += lowbit(idx)) bit[idx] += diff;
        }

        i64 point_query (int idx) const {
            return sum(idx) - sum(idx - 1);
        }

};

class rangeBIT {
    public:
        int n;
        BIT B1, B2;

        rangeBIT (int n = 0) { init(n); }

        void init (int m) {
            n = m;
            B1.init(n);
            B2.init(n);
        }

        i64 pre (i64 x) const {
            if (x <= 0) return 0;
            i64 s1 = B1.sum(x);
            i64 s2 = B2.sum(x);
            return (x + 1) * s1 - s2;
        }

        void range_add (int l, int r, i64 diff) {
            if (l > r) return;
            B1.point_add(l, diff);
            if (r + 1 <= n) B1.point_add(r + 1, -diff);
            B2.point_add(l, diff * l);
            if (r + 1 <= n) B2.point_add(r + 1, -diff * (r + 1));
        }
```

```
        i64 range_query (int l, int r) const {
            if (l > r) return 0;
            return pre(r) - pre(l - 1);
        }
};
```

线段树

```
class SegTree{
private:
    int n;
    vector <i64> tr, lz;

    void push_up (int p) {
        tr[p] = tr[p << 1] + tr[p << 1 | 1];
    }

    void apply (int p, int l, int r, i64 val) {
        tr[p] += val * (r - l + 1);
        lz[p] += val;
    }

    void push_down (int p, int l, int r) {
        if (lz[p] == 0) return;
        int mid = (l + r) >> 1;
        apply(p << 1, l, mid, lz[p]);
        apply(p << 1 | 1, mid + 1, r, lz[p]);
        lz[p] = 0;
    }

    void build (int p, int l, int r, const vector <i64> &a) {
        if (l == r) {
            tr[p] = a[l];
            return;
        }
        int mid = (l + r) >> 1;
        build(p << 1, l, mid, a);
        build(p << 1 | 1, mid + 1, r, a);
        push_up(p);
    }

    void modify (int p, int l, int r, int ql, int qr, i64 val) {
        if (ql <= l && r <= qr) {
            apply(p, l, r, val);
            return;
        }
        push_down(p, l, r);
        int mid = (l + r) >> 1;
        if (ql <= mid) modify(p << 1, l, mid, ql, qr, val);
        if (qr > mid) modify(p << 1 | 1, mid + 1, r, ql, qr, val);
        push_up(p);
    }

    i64 query (int p, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) {
```

```
            return tr[p];
        }
        push_down(p, l, r);
        int mid = (l + r) >> 1;
        i64 res = 0;
        if (ql <= mid) res += query(p << 1, l, mid, ql, qr);
        if (qr > mid) res += query(p << 1 | 1, mid + 1, r, ql, qr);
        return res;
    }

public:
    SegTree (int _n = 0) {
        init(_n);
    }

    void init (int _n) {
        n = _n;
        tr.assign(4 * n + 5, 0);
        lz.assign(4 * n + 5, 0);
    }

    void build (const vector <i64> &a) {
        build(1, 1, n, a);
    }

    void modify (int l, int r, i64 val) {
        modify(1, 1, n, l, r, val);
    }

    i64 query (int l, int r) {
        if (l > r) return 0;
        return query(1, 1, n, l, r);
    }
};
```

floyd

```
void solve ()
{
    int n, m; cin >> n >> m;
    vector <vector <i64> > dist(n + 5, vector <i64> (n + 5, 1e14));
    for (int i = 1; i <= m; i++) {
        int u, v, w; cin >> u >> v >> w;
        dist[u][v] = dist[v][u] = min(dist[u][v], w);
    }
    for (int i = 1; i <= n; i++) dist[i][i] = 0;

    for (int k = 1; k <= n; k++) {
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
            }
        }
    }

    for (int i = 1; i <= n; i++) {
```

```
        for (int j = 1; j <= n; j++) {
            cout << dist[i][j] << " \n"[j == n];
        }
    }
}
```

dijkstra

```
void solve ()
{
    int n, m, s; cin >> n >> m >> s;
    vector <vector <array<i64, 2> > > e(n + 1);
    for (int i = 1; i <= m; i++) {
        int u, v, w; cin >> u >> v >> w;
        e[u].push_back({v, w});
    }

    vector <i64> dis(n + 1, 1e14);
    priority_queue <array<i64, 2>, vector <array<i64, 2> > , greater<array<i64,
2> > > pq;

    pq.push({0, s}), dis[s] = 0;

    while (pq.size()) {
        auto [dist, u] = pq.top();
        pq.pop();
        if (dist != dis[u]) continue;
        for (auto [v, w] : e[u]) {
            if (dis[v] > dis[u] + w) {
                dis[v] = dis[u] + w;
                pq.push({dis[v], v});
            }
        }
    }

    for (int i = 1; i <= n; i++) {
        cout << dis[i] << ' ';
    }
}
```

最小生成树

```
void solve ()
{
    int n, m; cin >> n >> m;
    vector <array<i64, 3> > e;
    for (int i = 0; i < m; i++) {
        i64 u, v, w; cin >> u >> v >> w;
        e.push_back({w, u, v});
    }
    sort(e.begin(), e.end());

    DSU dsu(n);

    i64 ans = 0, cnt = 0;
```

```
    for (auto [w, u, v] : e) {
        u = dsu.find(u), v = dsu.find(v);
        if (u == v) continue;
        dsu.merge(u, v);
        ans += w;
        if (++cnt == n - 1) break;
    }

    if (cnt != n - 1) {
        cout << "orz" << '\n';
    }else {
        cout << ans << '\n';
    }

}
```

拓扑排序

```
void solve() {
    int n, m;
    cin >> n >> m;

    vector <vector <int> > e(n + 5);
    vector <int> ind(n + 5, 0);

    for (int i = 1; i <= m; i++) {
        int u, v;
        cin >> u >> v;
        e[u].push_back(v);
        ind[v]++;
    }

    queue <int> q;
    vector<int> tpsort;

    for (int i = 1; i <= n; i++) {
        if (!ind[i]) {
            q.push(i);
        }
    }

    if (q.empty()) {
        cout << -1 << '\n';
        return;
    }

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        tpsort.push_back(u);
        for (int v : e[u]) {
            ind[v]--;
            if (!ind[v]) {
                q.push(v);
            }
```

```
        }
    }

    if ((int)tpsort.size() != n) {
        cout << -1 << '\n';
        return;
    }

    for (int x : tpsort) {
        cout << x << ' ';
    }
    cout << '\n';
}
```

卡特兰数

```
vector <i64> f(n + 1);
f[0] = 1;
for (int i = 1; i <= n; i++) {
    f[i] = f[i - 1] * (4 * i - 2) / (i + 1);
}
```

ST倍增

```
void solve () // 查询区间最大/最小值
{
    i64 n, q; cin >> n >> q;
    vector <i64> v(n + 1);
    for (int i = 1; i <= n; i++) cin >> v[i];
    i64 t = __lg(n);
    vector <vector <i64> > stmx(n + 1, vector <i64> (30 + 1));
    vector <vector <i64> > stmn(n + 1, vector <i64> (30 + 1));
    for (int i = 1; i <= n; i++) stmx[i][0] = v[i], stmn[i][0] = v[i];
    for (int j = 1; j <= t; j++) {
        for (int i = 1; i <= n - (1LL << j) + 1; i++) {
            stmx[i][j] = max(stmx[i][j - 1], stmx[i + (1LL << (j - 1))][j - 1]);
            stmn[i][j] = min(stmn[i][j - 1], stmn[i + (1LL << (j - 1))][j - 1]);
        }
    }

    while (q--) {
        i64 l, r; cin >> l >> r;
        i64 len = __lg(r - l + 1);
        i64 mx = max(stmx[l][len], stmx[r - (1LL << len) + 1][len]);
        i64 mn = min(stmn[l][len], stmn[r - (1LL << len) + 1][len]);
        cout << mx - mn << '\n';
    }
}
```

LIS

```
void solve ()
{
```

```
    i64 n; cin >> n;
    vector <i64> v(n + 1);
    for (int i = 1; i <= n; i++) cin >> v[i];

    vector <i64> a(n + 1);
    a[1] = v[1];
    i64 ans = 1;
    for (int i = 2; i <= n; i++) {
        i64 pos = lower_bound(a.begin() + 1, a.begin() + 1 + ans, v[i]) -
a.begin();
        a[pos] = v[i];
        if (pos > ans) ans++;
    }

    cout << ans << '\n';
}
```

背包

01

```
    vector <i64> dp(m + 1, 0);

    for (int i = 1; i <= n; i++) {
        i64 cost, val;
        cin >> cost >> val;

        for (int j = m; j >= cost; j--) {
            dp[j] = max(dp[j], dp[j - cost] + val);
        }
    }
```

完全

```
    for (int i = 1; i <= n; i++) {
        for (int j = v[i]; j <= V; j++) {
            dp[j] = max(dp[j], dp[j - v[i]] + w[i]);
        }
    }
```

多重

```
    i64 N, W;
    cin >> N >> W;

    vector <i64> dp(W + 1, 1e16); // dp表示长度为W的时候的最小的数量cnt
    dp[0] = 0; // 1. 我们dp求的是最小值 2. 长度W不一定能够到达 -> 初始化dp[0] = 0, 其他
为INF

    for (int i = 0; i < N; ++i) {
        i64 w, cnt;
        cin >> w >> cnt;

        for (int k = 1; cnt > 0; k <<= 1) {
```

```
            i64 take = min(k, cnt);
            cnt -= take;

            // 二进制优化
            // 因为每个被拆分出来的组只能选择一次，相当于01背包
            i64 weight = take * w; // 这一组的总重量
            i64 cost = take;       // 这一组的总代价

            // 01背包，最小化版(逻辑都一样)
            for (int j = W; j >= weight; --j) {
                if (dp[j - weight] != 1e16) {
                    dp[j] = min(dp[j], dp[j - weight] + cost);
                }
            }
        }
    }

    cout << (dp[W] == 1e16 ? -1 : dp[W]) << '\n';
```

;acm

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;

void solve ()
{

}

int main ()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int _ = 1;
    // cin >> _;
    while (_--) {
        solve();
    }
    return 0;
}
```